# EECS3311 Fall 2026 Course Project Stage-1 Instruction

## Task 1: Define Your Agent Project

The goal of Stage 1 is to design your course project before implementation. Your project must be an AI agent-based software system, rather than a traditional application with only a simple LLM API call.

Provide a clear and detailed description of the proposed project, including its purpose, target users, major functionality, and how AI will be incorporated into the system.

Your proposed project must satisfy the following minimum requirements:

- **GUI**: The system must provide a graphical user interface through which users can interact with the major functionality of the application.
- **CLI:** The system must also provide a command-line interface through which users can access the major functionality of the agent.
- **At least 10 meaningful features**: The project must contain at least 10 distinct and non-trivial features. Simple operations such as Login, Logout, Exit, About, or Change Password do not count toward this requirement.
- **At least 5 design patterns are involved;**
- **AI model**: The system must integrate at least one AI/LLM model.
- **Agent behavior**: The AI component should participate in meaningful agent behavior, such as reasoning, planning, tool use, retrieval, memory, decision making, or multi-step task execution.

### 1.1 Project Description

Clearly describe:

- What problem does your project solve?
- Who are the target users?
- What can the agent do?
- Why is an AI agent appropriate for this problem?
- Which AI/LLM model(s) do you plan to use?
- How will the AI model interact with the rest of the software system?

### 1.2 Feature Specification

Identify and describe at least 10 major features.

For each feature, specify:

1. Feature ID and Name
2. Description — What does the feature do?
3. User Interaction — How does the user access or use this feature through the GUI?
4. Input — What information does the feature require?
5. Output — What does it produce?

6. AI Involvement — Is the feature deterministic, AI-based, or hybrid?
7. Expected Workflow — What happens when the feature is executed?
8. Error/Alternative Cases — What could go wrong and how should the system respond?

For example:

F01 — AI Trip Planning

The user enters a destination, dates, budget, and preferences through the GUI. The agent analyzes these constraints, retrieves relevant information through its available tools, and generates a multi-day itinerary. The itinerary is displayed in the GUI and can subsequently be modified by the user.

---

## Task 2: Design Your System Using UML

Use UML diagrams to make the architecture and behavior of your proposed system clear.

You must provide:

1. Class Diagram
2. Use-Case Diagram
3. Sequence Diagrams

The diagrams must be consistent with each other and with the features described in Task 1.

---

### 2.1 Class Diagram

Create a UML class diagram representing the major structure of your software.

The diagram should include:

- major classes and interfaces;
- important attributes;
- important methods;
- associations;
- dependencies;
- inheritance;
- aggregation/composition, where appropriate;
- multiplicities where meaningful.

Your class diagram should show both the traditional software components and the AI/agent components.

## Design Patterns

Your design must demonstrate the meaningful application of at least five design patterns covered in the course.

Possible patterns include:

- Strategy
- Observer
- Factory Method

- Abstract Factory
- Singleton
- Adapter
- Facade
- Command
- State
- Template Method
- Decorator
- Composite
- Iterator
- MVC

Do not add patterns simply to satisfy the numerical requirement. Each pattern should solve an identifiable design problem in your application.

For every pattern used, explain:

- What design problem does the pattern address?
- Which classes participate in the pattern?
- What role does each class play?
- Why is this pattern appropriate?
- What would be more difficult if the pattern were not used?

---

**2.2 Use-Case Diagram and Use-Case Descriptions**

Create a UML use-case diagram that represents the interactions between the system and its external actors.

Your use cases should collectively cover all major features of the project.

Identify all relevant actors, such as:

- primary user;
- administrator, if applicable;
- external services;
- external APIs/tools;
- AI services, where appropriate.

For each major use case, provide a description containing:

- Use Case ID;
- Use Case Name;
- Actor(s);
- Goal;
- Preconditions;

- Trigger;
- Main Success Scenario;
- Alternative/Exception Flows;
- Postconditions;
- Related Feature(s).

The use-case diagram and descriptions should make it possible to understand how users interact with all major functionality of your system.

---

**2.3 Sequence Diagrams**

Create sequence diagrams to show how objects collaborate at runtime to implement the major features/use cases.

Your sequence diagrams should show:

- the initiating actor;
- GUI/boundary objects;
- controller objects;
- domain objects;
- AI/agent components;
- external tools/services where applicable;
- important method calls;
- returned results;
- alternative/error flows where appropriate.

Your sequence diagrams should use the actual classes and important methods defined in your class diagram.

You do not necessarily need one sequence diagram for every individual feature if several closely related features share the same interaction structure. However, all important system behaviors must be represented by the submitted sequence diagrams.

---

**Task 3: Feature-to-Design Traceability**

A major goal of Stage 1 is to demonstrate that your design can actually support the functionality you proposed.

For every feature, explain exactly how the feature will be implemented by your proposed design.

Create a Feature-to-Design Traceability Table similar to the following:

| Feature | Description | Type | Related Use Case | Classes | Key Methods | Sequence Diagram | Design Pattern(s) |
|---|---|---|---|---|---|---|---|
| F01 | Generate travel plan | AI/Hybrid | UC01 Plan Trip | TripGUI, AgentController, Planner, LLMClient | createPlan(), generate(), invokeModel() | SD01 | Strategy, Facade |

| Feature | Description | Type | Related Use Case | Classes | Key Methods | Sequence Diagram | Design Pattern(s) |
|---|---|---|---|---|---|---|---|
| F02 | Modify itinerary | Hybrid | UC02 Modify Trip | TripGUI, ItineraryManager, AgentController | modifyTrip(), updateItinerary() | SD02 | Command |
| F03 | Save trip | Deterministic | UC03 Save Trip | TripRepository, Itinerary | save(), load() | SD03 | Repository/DAO |

For each feature, you should be able to answer:

Which classes implement this feature, which methods are called, which use case describes the user interaction, and which sequence diagram demonstrates the runtime interaction?

A feature should not appear only in the project description. It must be traceable to the software design.

---

**Task 4: Explain How Each Feature Is Realized**

In addition to the traceability table, provide a short explanation for each feature.

For example:

F01 — Generate a Travel Plan

Related Use Case: UC01 — Plan a Trip
Related Sequence Diagram: SD01 — Generate Trip Plan

Classes involved:

- TripPlanningGUI — collects destination, dates, budget, and preferences from the user.
- AgentController — coordinates the request.
- TripPlanner — constructs the overall planning process.
- ToolManager — manages tools available to the agent.
- SearchTool — retrieves relevant destination information.
- LLMClient — communicates with the selected LLM.
- Itinerary — represents the generated trip.

Important methods:

- TripPlanningGUI.submitRequest()
- AgentController.planTrip()
- TripPlanner.createPlan()
- ToolManager.executeTool()
- SearchTool.search()
- LLMClient.generate()

Execution:

When the user selects Generate Plan in the GUI, submitRequest() sends the user's constraints to AgentController. The controller invokes TripPlanner.createPlan(). The planner determines what information is

required and uses ToolManager to invoke appropriate tools. Relevant context is passed to LLMClient, which generates a proposed itinerary. The result is converted into an Itinerary object and returned to the GUI.

---

## Stage 1 Deliverables

Your Stage 1 submission should therefore contain:

1. Project Overview
    - Problem and motivation
    - Target users
    - Agent description
    - AI/LLM model(s)
    - Overall architecture
2. Detailed Feature Specifications
    - At least 10 meaningful features
    - Input/output and workflow for each feature
    - AI, deterministic, or hybrid classification
3. UML Class Diagram
    - Classes, interfaces, attributes, and methods
    - Relationships and multiplicities
    - AI/agent components
    - At least 5 meaningful design patterns
4. Design Pattern Explanations
    - Pattern
    - Participating classes
    - Problem addressed
    - Rationale for using it
5. Use-Case Diagram
    - Actors
    - Major use cases
    - Coverage of all major features
6. Detailed Use-Case Descriptions
7. Sequence Diagrams
    - Cover all important system interactions
    - Use classes/methods consistent with the class diagram
8. Feature-to-Design Traceability Table
    - Feature
    - Use case

- Classes
- Methods
- Sequence diagram
- Design pattern(s)

9. Feature Implementation Explanations
    - Explain how the proposed classes and methods collaborate to realize each feature.

## How to Submit

1. **Create a GitHub account/repository**
   **If you do not already have a GitHub account, create one.**
2. **Create a GitHub repository for your course project**
   **Create a dedicated repository for your EECS3311 course project. Use this same repository throughout all three stages of the project.**
3. **Upload your Stage 1 report**
   **Put your complete Stage 1 Project Design Report in the repository. The report should include all required descriptions, feature specifications, UML diagrams, design patterns, and feature-to-design mappings.**
4. **Submit your GitHub repository address**
   **Submit the URL of your EECS3311 course project GitHub repository through the course submission system.**

## Important

- **Make sure the repository is public.**
- **Do not create a new repository for each stage. Continue developing the same repository in Stages 2 and 3.**
- **Make sure your Stage 1 report is committed and pushed to GitHub before the submission deadline.**
- **Verify that the submitted repository URL works and points to the correct project.**

***** Some of the sample projects are listed below, you can also propose you own *****

**1. AI Travel Planning Agent**

**GUI:** Trip-planning dashboard with itinerary, map, activities, and saved trips.
**AI:** LLM agent plans trips, reasons over constraints, revises itineraries, and invokes travel-related tools.

**Features:** user profile and travel preferences; natural-language trip request; destination research; multi-day itinerary generation; attraction recommendations; restaurant recommendations; accommodation planning; transportation planning; budget estimation and tracking; schedule-conflict detection; weather-aware itinerary adjustment; itinerary editing through natural language; alternative-plan generation; packing-list generation; trip history; itinerary export.

**2. AI Personal Finance Assistant**

**GUI:** Financial dashboard with transactions, categories, budgets, charts, and AI chat.
**AI:** Agent analyzes financial records and explains spending patterns.

**Features:** import transactions; automatic transaction categorization; manual category correction; monthly budget creation; category-level budget tracking; spending trend analysis; unusual-expense detection; recurring-payment detection; subscription tracking; savings-goal management; natural-language financial queries; monthly summary generation; spending comparison across months; personalized saving suggestions; financial alerts; report export.

**3. AI Study and Learning Agent**

**GUI:** Course dashboard containing materials, study plans, quizzes, progress, and chat.
**AI:** LLM agent creates personalized learning activities from course materials.

**Features:** course creation; upload learning materials; document summarization; concept extraction; question answering over materials; personalized study-plan generation; flashcard generation; quiz generation; automatic quiz grading; answer explanation; weak-topic identification; adaptive question difficulty; progress tracking; study reminders; concept relationship visualization; learning-session history.

**4. AI Research Assistant**

**GUI:** Research workspace containing papers, notes, searches, summaries, and research questions.
**AI:** Agent searches, organizes, compares, and synthesizes research literature.

**Features:** define research topic; generate research questions; paper search; paper import; metadata extraction; paper summarization; contribution extraction; methodology extraction; result extraction; paper comparison; thematic paper grouping; related-paper recommendation; research-gap identification; citation management; research-note generation; literature-review outline generation; export research report.

**5. AI Software Project Manager**

**GUI:** Kanban board, project dashboard, task editor, risk panel, and agent interface.
**AI:** Agent decomposes requirements, assigns priorities, identifies risks, and updates plans.

**Features:** create projects; create/edit tasks; natural-language requirement entry; requirement-to-task decomposition; task prioritization; dependency management; deadline management; workload visualization; progress tracking; blocked-task detection; project-risk identification; sprint-plan generation; meeting-summary-to-task conversion; status-report generation; overdue-task alerts; project history.

**6. AI Customer Support Agent**

**GUI:** Customer chat, support-agent dashboard, ticket queue, knowledge-base viewer.
**AI:** Agent answers questions using a knowledge base and decides when human escalation is needed.

**Features:** customer chat; ticket creation; automatic ticket classification; priority prediction; knowledge-base retrieval; answer generation; source display; conversation summarization; sentiment/urgency detection; escalation to human support; suggested human-agent response; similar-ticket retrieval; ticket status management; customer history; unresolved-issue detection; support analytics; knowledge-gap identification.

**7. AI Job Search and Career Agent**

**GUI:** Job dashboard, job cards, skills profile, application tracker, and AI assistant.
**AI:** Agent compares candidate skills with job descriptions and helps manage applications.

**Features:** candidate profile; resume import; skill extraction; job import; job-description analysis; skill-gap identification; job matching; job comparison; customized resume suggestions; cover-letter drafting; interview-question generation; mock interview; answer feedback; application tracking; deadline tracking; company research notes; career-development plan.

**8. AI Event Planning Agent**

**GUI:** Event dashboard with schedule, attendees, tasks, venue information, and budget.
**AI:** Agent plans events subject to time, budget, capacity, and attendee constraints.

**Features:** create event; attendee management; venue requirements; agenda generation; session scheduling; speaker management; conflict detection; catering planning; budget management; task assignment; deadline tracking; invitation generation; attendee FAQ agent; schedule modification; contingency-plan generation; event checklist; post-event summary.

**9. AI Smart Recipe and Meal Planning Agent**

**GUI:** Weekly meal calendar, recipe browser, pantry, shopping list, and AI chat.
**AI:** Agent generates meal plans based on user constraints and available ingredients.

**Features:** preference profile; dietary-constraint configuration; pantry inventory; ingredient search; recipe recommendation; recipe generation; weekly meal planning; ingredient substitution; serving-size adjustment; nutrition-information organization; leftover utilization; shopping-list generation; duplicate-item consolidation; pantry updates; recipe rating; favorite recipes; natural-language meal-plan modification.

**10. AI News Analysis Agent**

**GUI:** News dashboard, topic feeds, article viewer, comparison panel, and timeline.
**AI:** Agent retrieves, clusters, summarizes, and compares coverage from multiple sources.

**Features:** topic creation; article retrieval; article categorization; article summarization; duplicate-story detection; event clustering; entity extraction; timeline generation; multiple-source comparison; claim extraction; source linking; topic tracking; personalized feed; saved articles; natural-language news queries; daily briefing generation; search history.

**11. AI Code Review Agent**

**GUI:** Repository/file browser, code viewer, review comments, issue panel, and AI interface.
**AI:** Agent analyzes code and generates structured review findings.

**Features:** repository import; file browsing; code selection; code summarization; issue detection; review-comment generation; severity categorization; duplicate-finding detection; coding-standard checking; potential-bug explanation; refactoring suggestions; test suggestion; review acceptance/rejection; human feedback capture; review-history tracking; report generation; comparison of reviews across versions.

**12. AI Test Generation and Debugging Agent**

**GUI:** Source-code browser, test explorer, execution console, failure dashboard, and AI agent panel.
**AI:** Agent generates tests, interprets failures, and proposes fixes.

**Features:** project import; source-code navigation; function/class selection; test generation; test editing; test execution; pass/fail visualization; exception analysis; failed-test explanation; boundary-case generation; regression-test management; coverage visualization; debugging suggestions; proposed patch generation; patch comparison; retesting; test-history tracking; testing report generation.

**13. AI Personal Knowledge Management Agent**

**GUI:** Notes workspace, document library, search, knowledge graph, and chat.
**AI:** Agent organizes and retrieves information from a user's personal document collection.

**Features:** create notes; upload documents; automatic tagging; topic extraction; document summarization; semantic search; natural-language Q&A; related-note discovery; duplicate-note detection; knowledge-graph generation; note linking; collection creation; task extraction; important-information highlighting; timeline creation; document comparison; knowledge summary generation.

**14. AI Email Management Agent**

**GUI:** Simulated inbox, email viewer, folders, tasks, and AI assistant.
**AI:** Agent classifies messages, summarizes threads, drafts responses, and extracts actions.

**Features:** receive/import emails; inbox search; email classification; priority identification; thread summarization; action-item extraction; deadline extraction; response drafting; response-tone selection; long-email summarization; related-email retrieval; spam/irrelevant-message classification; follow-up tracking; task creation from email; folder/tag management; daily inbox summary; response history.

**15. AI Meeting Assistant**

**GUI:** Meeting list, transcript viewer, notes, action-item board, and AI interface.
**AI:** Agent processes transcripts and converts discussions into structured knowledge and tasks.

**Features:** meeting creation; transcript import; speaker identification from structured transcripts; meeting summarization; topic segmentation; decision extraction; action-item extraction; task-owner identification; deadline extraction; unresolved-question detection;

follow-up generation; meeting search; related-meeting retrieval; decision history; task tracking; meeting comparison; minutes export.

**16. AI Shopping Comparison Agent**

**GUI:** Product search, comparison table, saved products, requirements panel, and chat.
**AI:** Agent interprets user requirements and compares products against them.

**Features:** natural-language requirements; product import/search; requirement extraction; product attribute extraction; comparison-table generation; filtering; sorting; budget constraints; feature matching; trade-off explanation; missing-information detection; alternative-product discovery; saved shortlist; product notes; price-history storage if data are available; user rating/preferences; comparison-report export.

**17. AI Home Maintenance Agent**

**GUI:** Home dashboard, rooms, appliances, maintenance calendar, tasks, and assistant.
**AI:** Agent interprets maintenance issues and creates maintenance plans.

**Features:** home profile; room management; appliance/equipment inventory; maintenance schedule; issue reporting; natural-language issue analysis; troubleshooting guidance; maintenance-task generation; priority classification; recurring-maintenance reminders; maintenance history; warranty-information storage; document/manual retrieval; parts/supply checklist; cost-record tracking; seasonal maintenance plan; maintenance report.

**18. AI Document Analysis and Contract Assistant**

**GUI:** Document library, document viewer, extracted-information panel, comparison screen, and chat.
**AI:** Agent extracts, summarizes, compares, and answers questions about documents. The application should clearly state that AI output is informational rather than professional legal advice.

**Features:** document upload; text extraction; document classification; section identification; document summarization; key-term extraction; obligation extraction; deadline extraction; party/entity extraction; clause search; natural-language document Q&A; cross-document comparison; inconsistent-term detection; missing-information identification; user annotation; document version management; structured report generation.

**19. AI Game Master / Interactive Story Agent**

**GUI:** Game screen, character panel, inventory, world map, event log, and conversation interface.
**AI:** Agent dynamically manages story progression while interacting with deterministic game state.

**Features:** player creation; character attributes; world creation; NPC management; natural-language actions; dynamic dialogue; inventory management; quest generation; quest tracking; location management; rule validation; combat/action resolution; persistent game state; story-memory management; consequence generation; save/load game; game-history visualization; ending generation.

**20. AI Campus Assistant**

**GUI:** Campus dashboard with courses, events, services, saved information, calendar, and chat.
**AI:** Agent answers questions using a provided campus knowledge base and helps students organize activities.

**Features:** student preference profile; campus information search; natural-language Q&A; course-information retrieval; course comparison; event discovery; event recommendation; campus-service discovery; building/location lookup; personalized schedule; deadline tracking; FAQ answering; document retrieval; favorite resources; reminder management; conflicting-event detection; daily campus summary; feedback mechanism.